

FIRE DETECTION SYSTEM

คู่มือการใช้งาน Mount Agent API

Local Control Interface และ Remote Device Protocol
รูปแบบ JSON สำหรับควบคุม Mount และรับข้อมูล Telemetry

Protocol version: 1.0

เอกสาร: ฉบับเริ่มต้นสำหรับนักพัฒนาระบบ

อัปเดต: 6 สิงหาคม 2026

1. ภาพรวม

Mount Agent เริ่มทำงานพร้อม fire-detector.service และเชื่อมคำสั่ง API เข้ากับ safety layer เดียวกับ Dashboard/ODriveMonitor เพื่อให้ motion limit, สถานะการเชื่อมต่อ Drive และเงื่อนไข enable ถูกตรวจในจุดเดียวกัน

Interface	การเชื่อมต่อ	วัตถุประสงค์	สถานะเริ่มต้น
Local Control Interface	Unix domain socket /run/fire-detector/mount-agent.sock	App ที่ทำงานในเครื่อง Mount เดียวกัน มี latency ต่ำและไม่ผ่าน network stack ภายนอก	เปิด
Remote Device Protocol	Mount เชื่อม TCP/TLS ออกไปหา Socket Receiver	ควบคุมและรับ telemetry จากศูนย์กลาง/LAN/Internet โดยไม่เปิด inbound port ที่ Mount	ปิดจนกว่าจะตั้งค่า credential

หลักสำคัญ: ทั้งสอง interface ใช้ JSON schema และชื่อคำสั่งเหมือนกัน แต่ Remote เพิ่ม TLS, device authentication และ command allowlist

Transport และ latency

- Client บนเครื่อง Mount ควรเปิด Unix domain socket ดังไว้หนึ่ง connection เป็นช่องทาง control ที่ overhead ต่ำสุดและไม่เปิด network port
- งานระยะไกลใช้ TCP/TLS connection แบบ persistent ที่ Mount เชื่อมออกไปหา Receiver จึงไม่ handshake ใหม่ทุกคำสั่งและใช้ได้หลัง router/CGNAT
- Local subscription ตั้งช่วงได้ 100–60,000 ms (สูงสุด 10 Hz); Remote telemetry ตั้ง telemetry_sec ได้ต่ำสุด 0.1 วินาที
- WebSocket /ws/status ของ dashboard เป็น read-only status stream ไม่ใช่ Mount Agent control transport

TCP เป็น byte stream: client ต้องสะสมข้อมูลจนพบ newline และห้ามสมมติว่า recv() หนึ่งครั้งคือ JSON หนึ่ง message ควรใช้ connection เดิมตลอด lease/control session

2. Message framing และโครงสร้าง JSON

Protocol ใช้ NDJSON: หนึ่ง JSON object ต่อหนึ่งบรรทัดและปิดท้ายด้วย newline (\n) ข้อความหนึ่งมีขนาดไม่เกิน 64 KiB

Command

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "cmd-0001",
  "action": "mount.get_status",
  "params": {}
}
```

Field	Required	คำอธิบาย
version	ใช่	ต้องเป็น 1.0
type	ใช่	คำสั่งต้องเป็น command
message_id	ใช่	ค่าที่ไม่ซ้ำ ใช้ตรวจคำสั่งซ้ำและจับคู่ response
action	ใช่	ชื่อคำสั่ง เช่น axis.goto
params	ใช่	JSON object; ใช้ {} เมื่อไม่มี parameter
control_lease_id	บางคำสั่ง	บังคับสำหรับคำสั่งเปลี่ยน state/เคลื่อนที่
expires_at	แนะนำ	ISO 8601 พร้อม timezone ปกป้องคำสั่งค้างมาทำงานภายหลัง

Response สำเร็จ

```
{
  "version": "1.0",
  "type": "response",
  "message_id": "rsp-a81...",
  "correlation_id": "cmd-0001",
  "action": "mount.get_status",
  "timestamp": "2026-07-23T15:23:19.632Z",
  "ok": true,
  "result": { "...": "..." }
}
```

Response ผิดพลาด

```
{
  "version": "1.0",
  "type": "response",
  "correlation_id": "cmd-0002",
  "action": "mount.goto",
  "ok": false,
  "error": {
    "code": "CONTROL_LEASE_REQUIRED",
    "message": "Acquire and maintain a control lease before sending this command.",
    "details": {}
  }
}
```

3. Control lease

ในหนึ่งช่วงเวลาจะมีผู้ควบคุมได้เพียงรายเดียว คำสั่งอ่านข้อมูลและคำสั่งหยุดฉุกเฉินไม่ต้องใช้ lease

1. เรียก `control.acquire` และเก็บ `lease_id`
2. แนบ `control_lease_id` กับคำสั่งควบคุม
3. เรียก `control.renew` ก่อน lease หมดอายุ
4. เมื่อจบงาน เรียก `control.release`

```
{"version": "1.0", "type": "command", "message_id": "lease-1",  
  "action": "control.acquire", "params": {"lease_ms": 5000}}
```

ช่วง lease ที่ระบบยอมรับคือ 500–30,000 ms หาก connection ปิด lease ของ session นั้นจะถูกปล่อยอัตโนมัติ

Fail-safe: `mount.stop`, `mount.disable`, `axis.stop` และ `axis.disable` เรียกได้โดยไม่ต้องมี lease เพื่อให้หยุดระบบได้เสมอ

4. รายการคำสั่ง System และสถานะ

Action	Lease	Params / ผลลัพธ์
<code>system.hello</code>	ไม่	คืน protocol version, device type และ capabilities
<code>system.ping</code>	ไม่	ตรวจว่า Agent ตอบสนอง พร้อม timestamp
<code>system.get_info</code>	ไม่	ข้อมูลระบบ/อุปกรณ์
<code>system.get_health</code>	ไม่	snapshot สุขภาพระบบและ Drive
<code>system.get_capabilities</code>	ไม่	รายการ action ที่ implementation รองรับ
<code>mount.get_status</code>	ไม่	snapshot ปัจจุบันของ Mount, axes, sensors และ errors

การตรวจ Run Mode ผ่าน API

ทุก snapshot จาก `mount.get_status`, `system.get_health`, REST GET `/api/status` และ WebSocket `/ws/status` มี field ต่อไปนี้เสมอ:

Field	Type	ความหมาย
<code>run_mode</code>	string	<code>simulation</code> เมื่อใช้โมเดลจำลอง หรือ <code>hardware</code> เมื่ออ่าน Drive/Encoder จริง
<code>simulation_mode</code>	boolean	<code>true</code> เฉพาะ Simulation Mode; <code>false</code> ใน Hardware Mode

หากต้องการอ่านเฉพาะโหมดโดยไม่รับ telemetry ทั้งหมด ให้เรียก GET `/api/system/run-mode`:

```
{  
  "ok": true,  
  "run_mode": "simulation",  
  "simulation_mode": true  
}
```

ข้อควรระวัง: Client ต้องตรวจสอบ run_mode ก่อนใช้ข้อมูลทดสอบหรือสั่งงาน ห้ามถือว่า connected: true หมายถึงเชื่อมต่อ Hardware จริง เพราะ Simulation Mode รายงาน connected เพื่อให้ทดสอบ API ได้ครบถ้วน

5. คำสั่ง Mount

Action	Lease	Parameters
mount.enable	ใช่	{ } เปิด closed-loop control ของแกนที่พร้อม
mount.disable	ไม่	{ } disable ทั้ง Mount
mount.stop	ไม่	{ } หยุดการเคลื่อนที่
mount.goto	ใช่	azimuth_deg, altitude_deg อย่างน้อยหนึ่งค่า และ optional max_velocity_deg_per_sec
mount.set_velocity	ใช่	azimuth_deg_per_sec, altitude_deg_per_sec อย่างน้อยหนึ่งค่า; ถ้าไม่เป็นศูนย์ต้องมี command_timeout_ms

ตัวอย่าง Goto สองแกน

```
{
  "version": "1.0", "type": "command", "message_id": "goto-1",
  "action": "mount.goto", "control_lease_id": "LEASE_ID",
  "expires_at": "2026-07-23T15:30:05.000Z",
  "params": {
    "azimuth_deg": 30.0,
    "altitude_deg": 15.0,
    "max_velocity_deg_per_sec": 10.0
  }
}
```

ตัวอย่าง Velocity พร้อม watchdog

```
{
  "version": "1.0", "type": "command", "message_id": "vel-1",
  "action": "mount.set_velocity", "control_lease_id": "LEASE_ID",
  "params": {
    "azimuth_deg_per_sec": 2.0,
    "altitude_deg_per_sec": 0.0,
    "command_timeout_ms": 500
  }
}
```

App ต้องส่งคำสั่ง velocity ใหม่ก่อน 500 ms หากขาดการสื่อสาร watchdog จะสั่งแกนนั้นเป็น 0 โดยอัตโนมัติ ช่วงที่อนุญาตคือ 100–5,000 ms

6. คำสั่งรายแกน

ค่า axis ใช้ azimuth/azm หรือ altitude/alt

Action	Lease	Parameters
axis.enable	ใช่	{"axis": "azimuth"}
axis.disable	ไม่	{"axis": "azimuth"}
axis.stop	ไม่	{"axis": "azimuth"}
axis.goto	ใช่	axis, position_deg, optional max_velocity_deg_per_sec
axis.set_velocity	ใช่	axis, velocity_deg_per_sec, command_timeout_ms

```
{"version": "1.0", "type": "command", "message_id": "axis-1",
  "action": "axis.goto", "control_lease_id": "LEASE_ID",
  "params": {"axis": "altitude", "position_deg": 20, "max_velocity_deg_per_sec": 5}}
```

7. Pointing, Calibration และ Tuning

Action	Parameters	หมายเหตุ
pointing.goto_altaz	เหมือน mount.goto	ต้องมี lease
calibration.start	axis, minimum_position_deg, maximum_position_deg, current_amp, velocity_deg_per_sec	ต้องมี lease และผ่าน motion/current limits
calibration.stop	axis	หยุด calibration
calibration.get_status	optional axis	อ่านสถานะ
tuning.apply	axis, values	ใช้ค่าชั่วคราว; ต้องมี lease
tuning.save	axis	เขียน configuration ลง Drive; ต้องมี lease

```
{
  "version": "1.0", "type": "command", "message_id": "cal-1",
  "action": "calibration.start", "control_lease_id": "LEASE_ID",
  "params": {
    "axis": "azimuth",
    "minimum_position_deg": -20,
    "maximum_position_deg": 20,
    "current_amp": 5,
    "velocity_deg_per_sec": 2
  }
}
```

คำเตือน: Calibration และ Tuning เปลี่ยนพฤติกรรมของ Drive ควรทำโดยผู้ดูแลในพื้นที่ปลอดภัย คำเริ่มต้นของ Remote allowlist ไม่อนุญาตคำสั่งกลุ่มนี้

8. Subscribe Telemetry

```
{"version": "1.0", "type": "command", "message_id": "sub-1",  
  "action": "subscribe",  
  "params": {"topics":  
    ["axis.telemetry", "sensor.status", "fault"], "interval_ms": 500}}
```

Agent ตอบรับหนึ่งครั้ง จากนั้นส่ง message ชนิด telemetry บน connection เดิม เรียก unsubscribe เพื่อหยุด

Topic	ข้อมูลหลัก
mount.status	ภาพรวม connected, health, timestamp และ axes
axis.telemetry	position, velocity, current, errors และสถานะแต่ละแกน
sensor.status	สถานะ CCW/CW limit sensor
drive.status	การเชื่อมต่อ, serial, bus power และ health
fault	รายการ drive error events
calibration.status	สถานะกระบวนการ Motor Calibration

9. การใช้งาน Local Control Interface

คำสั่งสำเร็จรูป

```
cd /home/pi/Public/Fire-Detector  
.venv/bin/python scripts/mount_agent_client.py system.hello  
.venv/bin/python scripts/mount_agent_client.py mount.get_status
```

ตัวอย่าง Python: connection เดียวพร้อม lease

```
import json, socket, uuid

s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
s.connect("/run/fire-detector/mount-agent.sock")

def call(action, params=None, lease_id=None):
    msg = {
        "version": "1.0", "type": "command",
        "message_id": str(uuid.uuid4()),
        "action": action, "params": params or {}
    }
    if lease_id:
        msg["control_lease_id"] = lease_id
    s.sendall(json.dumps(msg).encode() + b"\n")
    return json.loads(s.makefile("rb").readline())

lease = call("control.acquire", {"lease_ms": 5000})
lease_id = lease["result"]["lease_id"]
print(call("mount.get_status"))
# ตรวจสอบพื้นที่และความพร้อมก่อนปลด comment คำสั่งควบคุม
# print(call("mount.enable", lease_id=lease_id))
# print(call("axis.goto",
#           {"axis": "altitude", "position_deg": 10}, lease_id))
print(call("control.release", {"lease_id": lease_id}))
s.close()
```

Unix socket มี mode 660 และ Agent ตรวจสอบ peer UID โดยอนุญาต root หรือ UID เดียวกับ service เท่านั้น

10. Remote Device Protocol

Mount เป็น TCP client เชื่อมออกไปหา Socket Receiver จึงเหมาะกับระบบที่อยู่หลัง router/CGNAT ลำดับการเชื่อมต่อคือ:

1. TLS handshake และตรวจสอบ CA/hostname (ขั้นต่ำ TLS 1.2)
2. Mount ส่ง hello พร้อม Device ID และ capabilities
3. Server ส่ง auth_challenge ที่มี nonce
4. Mount ตอบ HMAC-SHA256 ของ device_id + nonce + timestamp
5. Server ส่ง auth_result; เมื่อผ่านจึงเริ่ม command/telemetry
6. Mount ส่ง heartbeat, telemetry และ reconnect แบบ exponential backoff เมื่อหลุด

ตัวอย่าง configuration

```
"mount_agent": {
  "local": {
    "enabled": true,
    "socket_path": "/run/fire-detector/mount-agent.sock",
    "mode": "660"
  },
  "remote": {
    "enabled": true,
    "host": "receiver.example.internal",
    "port": 15001,
    "server_name": "receiver.example.internal",
    "device_id": "NARIT-001",
    "ca_file": "/etc/fire-detector/ca.pem",
    "client_cert_file": "/etc/fire-detector/device.crt",
    "client_key_file": "/etc/fire-detector/device.key",
    "secret_file": "/etc/fire-detector/device.secret",
    "secret_env": "FIRE_DETECTOR_DEVICE_SECRET",
    "heartbeat_sec": 15,
    "telemetry_sec": 1
  }
}
```

Secret ต้องยาวอย่างน้อย 32 bytes และต้องมาจาก secret_file หรือ environment variable ห้ามเพิ่ม field secret ลงใน AppSetting.JSON

11. Camera Frame Ingest API

ระบบหลักไม่เปิดกล้อง Thermal หรือ Visible โดยตรง โมดูลกล้องภายนอกเป็นผู้เชื่อมต่อกล้องทั้งสอง แล้วส่งภาพ JPEG/PNG ที่ encode แล้วเข้ามาทาง HTTP API ระบบเก็บเฉพาะเฟรมล่าสุดของแต่ละกล้อง จึงไม่สะสมคิวเมื่อส่งประมาณ 60 FPS

วิธีแนะนำสำหรับ 60 FPS: Binary WebSocket

ห้ามส่ง byte array เป็น JSON array ของตัวเลขและไม่ควรใช้ Base64 สำหรับสตรีมความถี่สูง ให้เปิด connection เดียวไปที่ ws://192.168.1.105:8000/ws/camera-stream?role=producer&source=live แล้วส่งหนึ่ง binary message ต่อหนึ่งเฟรม:

```
[uint32 big-endian: ความยาว JSON header]
[JSON header แบบ UTF-8]
[JPEG/PNG bytes ที่เหลือทั้งหมด]
```

```
{"camera": "thermal", "mime_type": "image/jpeg",  
  "sha256": "64-lowercase-hex-characters",  
  "metadata": {"producer_sequence": 1234,  
               "captured_at": "2026-08-11T10:30:00.000Z",  
               "width": 640, "height": 512}}
```

byte ของภาพอยู่นอก JSON โดย WebSocket/TCP รักษาลำดับและส่งแบบ reliable, producer_sequence ใช้ตรวจช่องว่างจากต้นทาง และ sha256 ใช้ตรวจ payload ระดับ application

สำคัญ: ปิด WebSocket per-message compression (compression=None ใน Python) เพราะ JPEG/PNG บีบอัดมาแล้ว การบีบอัดซ้ำเปลือง CPU และทำให้อัตราเฟรมลดลงได้มาก

```
.venv/bin/python API-Doc/examples/04_publish_camera_binary_ws.py \  
  --camera thermal --frame thermal-frame.jpg --fps 60
```

HTTP compatibility API

กล้อง	Endpoint
Thermal	POST /api/camera-stream/frame/thermal?source=live
Visible	POST /api/camera-stream/frame/visible?source=live

ตัวอย่าง curl

```
curl -X POST "http://192.168.1.105:8000/api/camera-stream/frame/thermal?  
source=live" \  
  -H "Content-Type: image/jpeg" \  
  -H 'X-Camera-Metadata: {"captured_at": "2026-08-  
11T10:30:00.000Z", "width": 640, "height": 512, "producer_sequence": 1}' \  
  --data-binary @thermal-frame.jpg
```

Response เมื่อรับสำเร็จ

```
HTTP/1.1 202 Accepted  
Content-Type: application/json  
  
{"ok": true, "camera": "thermal", "source": "live", "sequence": 1234}
```

ตัวอย่าง HTTP แบบต่อเนื่อง

```
.venv/bin/python API-Doc/examples/03_publish_camera_frames.py \  
  --host 192.168.1.105 --port 8000 \  
  --camera thermal --frame thermal-frame.jpg --fps 60
```

- Content-Type ต้องเป็น image/jpeg หรือ image/png
- X-Camera-Metadata เป็น JSON object และไม่บังคับ
- ขนาดแต่ละเฟรมไม่เกิน 16 MiB และ endpoint ไม่รับ raw thermal sensor array
- รันอีก process ด้วย --camera visible เพื่อส่งกล้อง Visible พร้อมกัน
- ตรวจสอบสถานะที่ GET /api/camera-stream
- ดูผลที่ /api/camera-mjpeg/live/thermal และ /api/camera-mjpeg/live/visible

สำหรับ **production**: โมดูลกล้องควรส่งภาพที่ encode แล้วผ่าน LAN ที่เสถียร ตรวจสอบ HTTP status ทุกเฟรม และ reconnect/retry แบบมี backoff เมื่อระบบรับภาพไม่พร้อม

12. Error codes ที่ควรรองรับ

Code	ความหมาย/แนวทางจัดการ
UNSUPPORTED_VERSION	ใช้ protocol version ไม่ตรง
INVALID_JSON, INVALID_MESSAGE_TYPE	ตรวจสอบ JSON/framing/schema
MISSING_MESSAGE_ID	สร้าง ID ที่ไม่ซ้ำให้ทุกคำสั่ง
MESSAGE_TOO_LARGE	ลด payload ให้ต่ำกว่า 64 KiB
RATE_LIMITED	ลดเหลือไม่เกิน 20 commands/sec และทำ backoff
COMMAND_EXPIRED	ไม่ retry คำสั่งเดิม; สร้างคำสั่งใหม่หลังตรวจสอบ state
CONTROL_LEASE_BUSY	มี controller อื่นใช้งานอยู่ ห้ามแย่งควบคุม
CONTROL_LEASE_REQUIRED, CONTROL_LEASE_INVALID	acquire ใหม่และตรวจสอบ session
ACTION_NOT_ALLOWED	คำสั่งไม่อยู่ใน Remote allowlist
INVALID_PARAMETER	ตรวจสอบ parameter, motion limit และช่วงค่า
COMMAND_FAILED	อ่าน health/status ก่อน retry; หลีกเลี่ยง blind retry สำหรับ motion

13. Checklist สำหรับ App ที่ควบคุม Mount

- อ่าน mount.get_status และตรวจสอบ Drive/axis/limit ก่อน enable หรือ motion
- ใช้ connection เดียวตลอด control session และรักษา lease
- กำหนด expires_at สำหรับคำสั่ง motion จาก Remote
- velocity ต้องมี watchdog timeout และส่งคำสั่งศูนย์ก่อนจบ session
- มีปุ่ม Stop ที่เรียก mount.stop ได้โดยไม่ขึ้นกับ lease
- ใช้ message ID ไม่ซ้ำ และจับ response ด้วย correlation_id
- บันทึก audit log ฝั่ง Socket Receiver: user, device, action, parameters, result, timestamp
- ไม่เก็บ secret/private key ใน source control หรือหน้าเว็บ
- Remote server ต้องตรวจสอบสิทธิ์ผู้ใช้ก่อน forward คำสั่งทุกครั้ง

- ทดสอบด้วยความเร็ว/กระแสดำและพื้นที่ปลอดภัยก่อนใช้งานจริง

ไฟล์อ้างอิง implementation: `src/fire_detector/mount_agent.py`, `src/fire_detector/web_server.py` และ `scripts/mount_agent_client.py`

สถานะการ verify — 4 สิงหาคม 2026

Automated integration test ครอบคลุม Unix socket จริง, response correlation, duplicate protection, control lease และ telemetry subscription โดยไม่สั่ง hardware รันด้วย `PYTHONPATH=src .venv/bin/python -m unittest discover -s tests -v` ก่อน production ยังต้องทำ supervised hardware acceptance test เพื่อตรวจทิศทางแกน, software/physical limits, emergency stop, velocity watchdog และ Receiver reconnection